DESENVOLVIMENTO DE API BACK-END · ENGENHARIA DE SOFTWARE · 6º PERÍODO

Semana 04
Persistência, Flyway e erros padronizados

Apostila de estudo: teoria, exemplos, construção com IA e exercícios

Até aqui a API respondia, mas não guardava nada de forma confiável. Nesta semana ela ganha um banco de verdade. Você vai entender como objetos Java viram linhas de tabela, por que o schema do banco é versionado como código, o que uma transação garante e por que o cliente nunca deve receber a entidade. No fim, todos os erros da API passam a sair num único formato, o ProblemDetail. Leia antes ou depois da aula; os exercícios do fim ajudam a conferir o que ficou.

Conteúdo

1. Objetivos da semana
2. Objetos não são tabelas
3. JPA, Hibernate e Spring Data
4. Ciclo de vida da entidade
5. Mapeamentos e relacionamentos
6. Transações e @Transactional
7. Migrações com Flyway
8. Repository e Data Mapper
9. DTO: o que sai da API
10. Exceções e ProblemDetail
11. Paginação
12. Health e o primeiro teste
13. Aplicação no ServiceHub
14. Construindo com IA
15. Glossário
16. Exercícios
17. Para estudar

1. Objetivos da semana

↑ conteúdo

Ao final desta semana você deve ser capaz de:

1. explicar a impedância objeto-relacional e o papel de um ORM, distinguindo JPA (especificação), Hibernate (implementação) e Spring Data JPA (repositórios);
2. descrever o ciclo de vida de uma entidade, o contexto de persistência e o efeito de LAZY, EAGER e do problema N+1;
3. usar transações com `@Transactional` e justificar `readOnly` e `open-in-view=false`;
4. versionar o schema com migrações Flyway, sabendo por que uma migração aplicada nunca é editada e por que o Hibernate só valida;
5. devolver DTOs paginados e padronizar os erros com `@RestControllerAdvice` e ProblemDetail, provando com um `@WebMvcTest`.

2. Objetos não são tabelas

↑ conteúdo

Um programa Java pensa em **objetos** que apontam uns para os outros na memória. Um banco relacional pensa em **linhas** e **colunas**, ligadas por chaves. Os dois modelos descrevem o mesmo mundo, mas com regras diferentes. Essa diferença tem nome: **impedância objeto-relacional** (*object-relational impedance mismatch*). O termo vem da engenharia elétrica, onde dois circuitos com impedâncias diferentes perdem energia ao se ligar.

Onde os dois modelos divergem

Aspecto	Objetos (Java)	Relacional (SQL)
Identidade	Referência na memória; igualdade por `equals()`	Chave primária
Associação	`livro.getAutor()` devolve um objeto `Autor`	Coluna `autor_id` e um `JOIN`
Coleções	Uma lista dentro do objeto	Linhas em outra tabela apontando de volta
Herança	Classe e subclasses	Não existe; precisa de uma estratégia de mapeamento
Tipos	`enum`, `BigDecimal`, `LocalDateTime`	`VARCHAR`, `NUMERIC`, `TIMESTAMP`

O que um ORM faz

Sem ajuda, cada consulta exige escrever o SQL, executar pelo JDBC, ler o resultado coluna por coluna e montar o objeto à mão. Para gravar, o caminho inverso. Um **ORM** (*Object-Relational*

Mapping, mapeamento objeto-relacional) automatiza essa tradução: você descreve uma vez como a classe corresponde à tabela, e ele gera o SQL de leitura e gravação.

Em vez de um laço que lê `rs.getLong("id")` e `rs.getString("titulo")` e preenche um `Livro` à mão, você anota a classe com `@Entity` e `@Table(name = "livros")`, e uma chamada como `repo.findAll()` devolve a lista pronta. Um nome de coluna errado deixa de ser descoberto só em execução, no meio de uma consulta.

CONFUSÃO COMUM

O ORM não elimina a impedância; ele só automatiza a tradução. Quem não entende as duas pontas cai em armadilhas como o N+1 (seção 5). E o ORM não dispensa saber SQL: as migrações desta semana são SQL puro.

3. JPA, Hibernate e Spring Data

↑ conteúdo

Três nomes aparecem juntos e costumam ser confundidos. Eles ocupam camadas diferentes.

- **JPA** (Jakarta Persistence) é uma *especificação*: anotações (`@Entity`, `@Id`, `@Column`) e interfaces (`EntityManager`) no pacote `jakarta.persistence`. Como uma norma técnica de tomada, diz o formato, mas não fabrica nada.
- **Hibernate** é uma *implementação* da especificação: gera e executa o SQL. Outra implementação conhecida é o EclipseLink.
- **Spring Data JPA** fica uma camada acima: você declara uma interface de repositório e o Spring cria a implementação em tempo de execução.

Por que separar especificação de implementação? Porque o código da aplicação depende só das anotações padronizadas. Em tese, trocar o Hibernate por outra implementação não exige reescrever as entidades. Na prática, quase todo projeto Spring usa o Hibernate, que já vem com o starter de JPA.

PEGADINHA DE VERSÃO

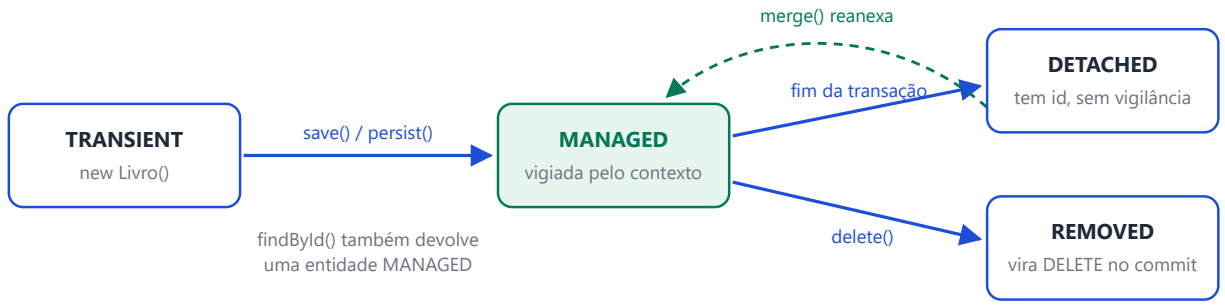
Quando a especificação fazia parte do Java EE, o pacote se chamava `javax.persistence`. Depois que ela passou para a Eclipse Foundation, com o nome Jakarta EE, o pacote mudou para `jakarta.persistence`. Spring Boot 3 em diante só aceita `jakarta`. Tutoriais antigos e código gerado por IA ainda trazem `javax`: o projeto nem compila.

No ServiceHub, o `pom.xml` tem `spring-boot-starter-data-jpa` (traz Hibernate e Spring Data), `spring-boot-starter-flyway` mais `flyway-database-postgresql` (migrações) e o driver do PostgreSQL 16. Lembre a pegadinha do Boot 4 vista na Semana 01: a biblioteca sem o starter não ativa a auto-configuração.

4. Ciclo de vida da entidade

↑ conteúdo

Uma **entidade** é uma classe anotada com `@Entity` cujas instâncias correspondem a linhas de uma tabela. Cada instância passa por quatro estados, e o estado define se as mudanças no objeto chegam ao banco.



Os quatro estados de uma entidade JPA. Só no estado MANAGED as alterações viram UPDATE automaticamente.

Em resumo: um objeto **transient** (na especificação, *new*) existe só na memória; um **managed** tem as mudanças detectadas e gravadas sozinhas no commit; um **detached** tem id, mas suas mudanças não chegam ao banco a menos que seja reanexado; um **removed** vira `DELETE` no commit.

Contexto de persistência e dirty checking

O **contexto de persistência** é o conjunto de entidades gerenciadas durante uma transação. Ele dá duas garantias. Primeiro, existe uma única instância por id: buscar o livro 7 duas vezes na mesma transação devolve o mesmo objeto, e a segunda busca nem vai ao banco. Segundo, o contexto guarda uma cópia do estado lido e, antes do commit, compara cada entidade com essa cópia. O que mudou vira `UPDATE`. Essa comparação se chama **dirty checking** (verificação de sujeira).

```
@Transactional
public void renomear(Long id, String novoTitulo) {
    Livro livro = repo.findById(id).orElseThrow();  // MANAGED
    livro.setTitulo(novoTitulo);                    // só muda o objeto
}   // commit: o Hibernate percebe a mudança e executa o UPDATE
```

LIGAÇÃO COM O SERVICEHUB

A exclusão é lógica (ADR-0002): "apagar" um serviço é mudar `active` para `false` numa entidade gerenciada. É exatamente o que faz o `deleteService` do `ServiceOfferingService`: ele não chama `save()`, e o commit gera o `UPDATE`.

5. Mapeamentos e relacionamentos

↑ conteúdo

Mapeando colunas

- `@Entity` e `@Table(name = "livros")` marcam a classe e dizem a tabela.
- `@Id` com `@GeneratedValue(strategy = IDENTITY)`: chave gerada pelo banco, que combina com `BIGSERIAL` do Postgres.
- `@Column(name = "data_publicacao", nullable = false)` liga atributo e coluna quando os nomes diferem e declara restrições.
- `@Enumerated(EnumType.STRING)` grava o *nome* do enum (`"DISPONIVEL"`), e não sua posição.

POR QUE NUNCA ORDINAL

Com `EnumType.ORDINAL` o banco guarda 0, 1, 2... Se alguém inserir um valor no meio do enum Java, todas as linhas já gravadas mudam de significado sem nenhum erro. O Mapa exige `EnumType.STRING`.

Relacionamentos e quando carregar

Um livro tem um autor; um autor tem muitos livros. No Java isso vira `@ManyToOne` de um lado e, se fizer sentido, `@OneToMany` do outro. O atributo `fetch` diz *quando* o objeto relacionado é buscado no banco:

Fetch	Quando carrega	Padrão do JPA em
LAZY	Só quando o código acessa o campo (o ORM coloca no lugar um objeto substituto, o *proxy*).	`@OneToMany` e `@ManyToMany`
EAGER	Sempre, junto com a entidade.	`@ManyToOne` e `@OneToOne`

EAGER parece cômodo, mas carrega o grafo inteiro em toda consulta, inclusive quando você não precisa. A prática recomendada é declarar `fetch = FetchType.LAZY` também nos `@ManyToOne` e decidir, consulta a consulta, quando trazer o relacionado junto. O preço do LAZY é a `LazyInitializationException`: acessar o campo depois que a transação fechou, quando já não há sessão para buscar o dado.

O problema N+1

Imagine listar 20 livros mostrando o nome do autor. Uma consulta traz os 20 livros. Depois, para cada livro, o acesso a `getAutor().getNome()` dispara mais uma consulta. Resultado: 1 + 20 idas ao banco, onde uma bastava.

```
select ... from livros limit 20
select ... from autores where id = ?    -- livro 1
select ... from autores where id = ?    -- livro 2
...                                     -- mais 18 vezes
```

O N+1 não dá erro. A tela funciona, e só fica lenta quando a tabela cresce. Por isso a melhor ferramenta é o log do SQL gerado: conte os `SELECT`s. As duas soluções mais comuns trazem a associação na mesma consulta, com `JOIN`:

@EntityGraph

```
@EntityGraph(attributePaths = "autor")
Page<Livro> findAll(Pageable p);
```

Declara, no método do repositório, quais associações vêm junto.

JOIN FETCH

```
@Query("select l from Livro l
        join fetch l.autor")
List<Livro> comAutor();
```

Escreve a junção na própria consulta JPQL.

No ServiceHub, o `ServiceOffering` tem `@ManyToOne(fetch = LAZY)` com o prestador (`User`), e o repositório usa `@EntityGraph(attributePaths = "provider")` na listagem, porque a resposta mostra `providerName`. A seção 13 mostra o código.

6. Transações e @Transactional

↑ conteúdo

Uma **transação** é um grupo de operações que o banco trata como uma unidade. O exemplo clássico é a transferência bancária: debitar a conta A e creditar a conta B. Se só a primeira acontecer, o dinheiro some. As garantias de uma transação são resumidas pela sigla **ACID**:

Garantia	O que significa
Atomicidade	Tudo ou nada. Se algo falha no meio, o banco desfaz o que foi feito (*rollback*).
Consistência	A transação leva o banco de um estado válido a outro: `NOT NULL`, `CHECK` e chaves estrangeiras continuam valendo.
Isolamento	Transações simultâneas não enxergam o trabalho pela metade uma da outra, no grau definido pelo nível de isolamento (no Postgres, o padrão é *read committed*).
Durabilidade	Depois do *commit*, o dado sobrevive a uma queda do servidor.

Como o Spring aplica a transação

Com `@Transactional` num método, o Spring não muda o seu código: ele coloca um **proxy** (um objeto intermediário) na frente do bean. Quem chama o service fala, sem saber, com o proxy.

O proxy abre a transação antes do método e, depois dele, faz *commit* ou *rollback*. A anotação fica no service, a camada que conhece a regra de negócio inteira, e não no controller nem no repositório.

- **Rollback por padrão:** exceções *unchecked* (`RuntimeException` e `Error`) desfazem a transação; exceções *checked*, por padrão, *não* desfazem. É mais um motivo para as exceções de negócio serem unchecked (seção 10).
- **Autochamada não passa pelo proxy:** se um método da classe chama outro método da mesma classe (`this.outro()`), a anotação do segundo é ignorada.
- **`readOnly = true` :** nas leituras, avisa que nada será gravado. O Hibernate deixa de fazer o dirty checking e o *flush* no fim, o que poupa trabalho, e a intenção fica explícita no código.

open-in-view=false

Por padrão, o Spring Boot mantém a sessão do Hibernate aberta até a resposta HTTP ser escrita (*Open Session in View*) e avisa isso no log da subida. Parece conveniente: qualquer campo LAZY pode ser lido no controller ou na serialização para JSON. O custo é escondido: consultas disparadas fora do service, conexões presas por mais tempo e N+1 durante a serialização. Com `spring.jpa.open-in-view=false` , a sessão fecha junto com a transação. A regra passa a ser clara: **o DTO é montado dentro do método transacional do service**.

No ServiceHub, desativar um usuário também desativa os serviços dele (RN11), no mesmo método `@Transactional` : se a segunda gravação falhar, a primeira é desfeita.

7. Migrações com Flyway

↑ conteúdo

Sem controle, cada banco evolui à mão e diverge: "na minha máquina a coluna existe". Uma **migração** é um script que leva o schema de uma versão à seguinte. As migrações são aplicadas em ordem e registradas numa tabela de histórico. Assim, toda máquina, todo teste e todo container sobem o mesmo banco, na mesma sequência. O paralelo é com o Git: migração está para o banco como o commit está para o código.



O que o Flyway faz a cada subida da aplicação.

Nome e tipos

O nome segue a forma `V{versão}__{descrição}.sql`: V maiúsculo, versão crescente, **dois** sublinhados e uma descrição com sublinhados no lugar de espaços. Um sublinhado só, como em `V2_Create.sql`, faz o arquivo não ser reconhecido como migração.

Tipo	Exemplo	Quando roda
Versionada	`V3__add_isbn.sql`	Uma única vez, na ordem da versão. Usada para tabelas e colunas.
Repetível	`R__views.sql`	De novo sempre que o conteúdo muda, depois das versionadas. Usada para views e funções.

Checksum: por que migração aplicada não se edita

Ao aplicar uma migração, o Flyway grava na `flyway_schema_history` a versão, a descrição e um **checksum**: um número calculado a partir do conteúdo do arquivo. Mudou um caractere, muda o checksum. Na subida seguinte, o Flyway recalcula e compara. Se a V2 foi editada depois de aplicada, a validação falha e a aplicação não sobe. Isso não é um defeito: é a proteção que impede bancos diferentes de terem "a mesma V2" com conteúdos diferentes. **Errou? Crie uma versão nova** que corrige.

ddl-auto=validate: o Hibernate só confere

O Hibernate sabe criar e alterar tabelas sozinho, conforme `spring.jpa.hibernate.ddl-auto`: `create` e `create-drop` apagam e recriam o schema a cada subida (protótipos descartáveis); `update` tenta alterar o schema para bater com as entidades, sem nunca remover colunas e sem gerar migração; `validate` só compara entidades e tabelas e impede a subida se algo não bate; `none` não faz nada. O ServiceHub usa `validate`.

Com Flyway, quem cria e altera o schema é o Flyway; o Hibernate só valida. A combinação dá um sinal precoce e preciso: mudou a entidade e esqueceu a migração, a subida falha dizendo qual coluna falta.

> **CONFUSÃO COMUM**
>
> "Coloque `ddl-auto=update` para facilitar" é uma sugestão frequente em código gerado por IA. A aplicação sobe, mas o schema real deixa de ser descrito pelas migrações, e o próximo banco criado do zero (o do teste, o do colega, o de produção) fica diferente.

Como as migrações usam SQL do Postgres (`BIGSERIAL`, `CHECK`, `ALTER TABLE ... DROP CONSTRAINT`), os testes do projeto também usam Postgres real, via Testcontainers, e não um banco em memória como o H2 (ADR-0005).

8. Repository e Data Mapper

↑ conteúdo

Martin Fowler catalogou formas de ligar objetos ao banco. A pergunta central é: *a entidade sabe se gravar, ou outra peça faz isso por ela?*

Padrão	Ideia	Exemplo
Active Record	O próprio objeto tem `save()` e `find()`; dado, regra e SQL moram juntos. Comum em frameworks como Rails e Laravel.	`livro.save()`
Data Mapper	Uma camada separada move dados entre objeto e tabela. A entidade não conhece o banco.	O Hibernate, por baixo do JPA
Repository	Uma interface que parece uma coleção de objetos do domínio, com buscas na linguagem do negócio.	`livroRepository.findByIsbn(...)`

No ServiceHub as duas ideias se somam: o service conversa com um Repository (a interface do Spring Data), e o Hibernate faz o papel de Data Mapper. A entidade `User` não tem uma linha de SQL. A vantagem aparece nos testes: para testar uma regra do service sem banco, troca-se o repositório por um dublê.

Repositórios do Spring Data

Você declara uma interface que estende `JpaRepository<Entidade, TipoDoId>` e ganha `save`, `findById`, `findAll`, `findAll(Pageable)` e `delete`. Métodos com nomes no padrão `findBy…`, `existsBy…` e `countBy…` viram consultas derivadas do nome. O que o nome não expressa bem vira `@Query` em JPQL, sempre com parâmetros (`:email`).

> **SEGURANÇA**
> Nunca concatene texto do usuário numa consulta (`"... where titulo = '" + q + "'"`): é a porta da injeção de SQL.

9. DTO: o que sai da API

↑ conteúdo

Um **DTO** (*Data Transfer Object*, objeto de transferência de dados) só carrega dados entre processos, sem regra de negócio. Fowler o descreve para reduzir chamadas remotas; numa API REST ele tem um papel a mais: separar o **contrato público** do **modelo interno**. A entidade é detalhe de implementação; o DTO é contrato.

Devolvendo a entidade

```
@GetMapping("/{id}")
public Conta buscar(@PathVariable Long id)
{
  return contaService.buscar(id);
}
```

Sai tudo: hash de senha, saldo, campos internos. E todo campo novo da entidade vira campo público sem ninguém decidir.

Devolvendo um DTO

```
public record ContaResponse(
    String numero, String titular) {
  static ContaResponse from(Conta c) {
    return new ContaResponse(
        c.getNumero(), c.getTitular());
  }
}
```

Sai só o que foi escolhido. A entidade pode mudar por dentro sem quebrar quem consome.

O DTO também protege na entrada. Se o controller aceitasse a entidade no corpo, o cliente poderia mandar `"role": "ADMIN"` ou `"active": true` e o ORM gravaria. Esse ataque se chama **mass assignment** (atribuição em massa). Com um DTO de entrada que simplesmente não tem esses campos, o problema não existe. Expor campos demais e aceitar campos demais são, juntos, o item API3 da lista OWASP de segurança de APIs, assunto da Semana 08.

Records como DTO

O `record` do Java 21 é ótimo para DTO: é imutável, declara os campos numa linha e gera construtor, acessores, `equals`, `hashCode` e `toString`. O custo do DTO é o mapeamento entidade → DTO, que fica no service, dentro da transação (seção 6). Na criação, vale o contrato da Semana 02: **201 Created**, o DTO no corpo e o header `Location`, escrito `ResponseEntity.created(uri).body(dto)`.

10. Exceções e ProblemDetail

↑ conteúdo

Checked e unchecked

Uma **exceção** é um objeto que interrompe o fluxo normal e sobe pela pilha de chamadas até alguém tratá-la. Em Java, o ramo da hierarquia decide se o compilador obriga a tratá-la:

Tipo	Compilador obriga?	Exemplo	Quando usar
Checked (`Exception`)	Sim: `try/catch` ou `throws`	`IOException`	Falha externa que o chamador pode resolver.
Unchecked (`RuntimeException`)	Não	`IllegalArgumentException`	Erro de programação ou regra de negócio violada.
`Error`	Não, e não se trata	`OutOfMemoryError`	Falha da própria JVM.

Numa API, as exceções de negócio costumam ser unchecked: sobem do service até um tratador global sem encher as assinaturas de `throws`, e desfazem a transação por padrão. O que *não* se faz é lançar `RuntimeException` crua: o tratador não sabe que status dar, e o cliente recebe 500 quando deveria receber 404 ou 409. Crie exceções com nome de domínio.

ERRO COMUM EM CÓDIGO GERADO

`catch (Exception e) { }` vazio: engole o erro, a transação faz commit do que estiver pela metade e o bug some do log.

Um formato só: ProblemDetail (RFC 9457)

Se cada projeto inventa o seu JSON de erro, cada cliente precisa aprender um contrato novo. A **RFC 9457** (*Problem Details for HTTP APIs*, de 2023, que substitui a RFC 7807) padroniza esse formato, com o tipo de mídia `application/problem+json`:

- `type`: URI que identifica o tipo de problema. É estável, e o cliente decide por ele. Na ausência, vale `about:blank`.
- `title`: resumo curto para gente ler, igual em todas as ocorrências do mesmo tipo.
- `status`: o status HTTP, repetido no corpo; `detail`: a explicação desta ocorrência; `instance`: a URI desta ocorrência, em geral o caminho chamado.

A RFC permite **membros de extensão**. O ServiceHub acrescenta `timestamp` em todos e `errors: [{field, message}]` nos erros de validação.

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{ "type": "urn:servicehub:problem:not-found",
  "title": "Recurso não encontrado",
  "status": 404,
  "detail": "Serviço não encontrado: 99",
  "instance": "/api/v1/services/99",
  "timestamp": "2026-11-05T17:30:00Z" }
```

Tratamento global com @RestControllerAdvice

Em vez de `try/catch` em cada controller, uma única classe anotada com `@RestControllerAdvice` traduz exceções em respostas. Cada método `@ExceptionHandler(Tipo.class)` cuida de um tipo de exceção. O Spring já tem a classe `ProblemDetail` (desde o Spring Framework 6). Estendendo `ResponseEntityExceptionHandler`, os erros que o próprio Spring MVC detecta (JSON malformado, método não permitido, tipo de conteúdo não suportado) também saem no mesmo formato.

Situação no ServiceHub	Status	type
`ResourceNotFoundException`	404	`…:not-found`

Situação no ServiceHub	Status	type
`EmailAlreadyExistsException`, `BusinessRuleException`	409	`…:email-already-exists`, `…:business-rule`
`@Valid` reprovou um DTO	400 + `errors`	`…:validation-error`
Qualquer outra exceção	500	`…:internal-error`, sem stack trace

O 500 nunca mostra stack trace nem mensagem interna (`server.error.include-stacktrace=never`). O erro real vai só para o log, e o `detail` pede ao cliente o `X-Request-Id` para o suporte localizar a ocorrência.

11. Paginação

↑ conteúdo

Tabelas crescem. Uma listagem sem limite um dia devolve milhares de linhas, gasta memória no servidor e trava o cliente. **Paginar** é devolver um pedaço por vez. Há duas formas principais. Na paginação por **offset** (`?page=3&size=20`), o SQL é `ORDER BY id LIMIT 20 OFFSET 60`: dá para pular para qualquer página, mas páginas distantes ficam lentas, porque o banco lê e descarta as linhas puladas, e inserções no meio podem repetir ou pular itens. Na paginação por **keyset** ou cursor (`?after=120&size=20`), o SQL é `WHERE id > 120 ORDER BY id LIMIT 20`: só existe "a próxima página", mas o custo é o mesmo em qualquer ponto e o resultado é estável.

Keyset é a escolha para feeds de rolagem infinita e tabelas muito grandes. Para um catálogo, o offset é simples e suficiente; é o que o ServiceHub usa, com o `Pageable` do Spring Data. Para informar o total de páginas, o Spring Data faz um `COUNT` extra sozinho.

Pageable, teto e formato

O controller recebe um `Pageable` montado a partir de `page`, `size` e `sort` da query string, com padrões dados por `@PageableDefault`. O repositório devolve um `Page<T>`, e `page.map(...)` converte cada entidade no DTO. Duas propriedades fecham o contrato:

```
spring.data.web.pageable.max-page-size=100
spring.data.web.pageable.serialization-mode=via-dto
```

- **Teto de 100:** um `size=500` devolve 100 itens, nunca erro. O teto é regra de segurança, e não só de desempenho: sem ele, um único pedido pode puxar a tabela inteira e esgotar a memória.
- **`via-dto`:** gera o JSON estável `{content, page{size, number, totalElements, totalPages}}`, em vez de serializar os detalhes internos da implementação de `Page`.

PEGADINHA

Não adicione `@EnableSpringDataWebSupport`. No Spring Boot essa anotação desliga a configuração automática de paginação, e o teto de 100 itens some sem aviso.

Outro cuidado: `sort` aceita qualquer caminho de propriedade da entidade. Sem uma lista de campos permitidos, alguém poderia ordenar por `passwordHash`. O ServiceHub restringe com o `SortGuard`.

12. Health e o primeiro teste

↑ conteúdo

/actuator/health

O **Actuator** (starter `spring-boot-starter-actuator`) expõe endpoints de operação. O mais usado é `/actuator/health`, que responde `{"status":"UP"}` e inclui a conexão com o banco: com o Postgres fora do ar, vira `DOWN`. Ele será usado pelo `HEALTHCHECK` do Dockerfile (Semana 11), pelo Compose (Semana 12) e pelo CI (Semana 14).

@WebMvcTest: só a camada web

Um teste de **fatia web** sobe só o que atende HTTP: controllers, o `@RestControllerAdvice`, a conversão de JSON e a validação. Services e repositórios não são carregados; no lugar do service entra um dublê do Mockito. Sem banco, o teste roda em segundos e prova o contrato da borda: status, formato do erro, campo apontado.

No Spring Boot 4, o dublê é declarado com `@MockitoBean` (o antigo `@MockBean` foi removido), e o pacote do `@WebMvcTest` mudou para `org.springframework.boot.webmvc.test.autoconfigure`.

13. Aplicação no ServiceHub

↑ conteúdo

Os trechos abaixo são do código real do projeto. Cortes estão marcados como "resumido".

Migrações: corrigir para a frente

A `V2__Create_services_table.sql` do projeto nasceu sem `updated_at`, sem os `CHECK` e sem o `ON DELETE RESTRICT` do schema final. Nada foi corrigido editando a V2: a V4 trocou a chave estrangeira, a V6 removeu um índice redundante e a V9 alinhou tudo a `docs/alunos/servicehub_schema.sql`. A V3 se chama `Create_users_table` embora só acrescente colunas: migração aplicada não se renomeia.

A entidade

`model/ServiceOffering.java` (resumido; o nome evita colidir com a anotação `@Service`):

```
@Data @Entity @Table(name = "services")
public class ServiceOffering {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;
    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private ServiceCategory category;
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "provider_id", nullable = false)
    private User provider;
    @Column(name = "is_active")
    private boolean active = true;
    // title, description, createdAt, updatedAt e callbacks omitidos
}
```

Repare: `BigDecimal` para dinheiro (nunca `double`), `EnumType.STRING`, LAZY no prestador e `name = "is_active"` porque coluna e atributo têm nomes diferentes.

Repositório e service

O `ServiceOfferingRepository` sobrescreve `findAll(Specification, Pageable)` com `@EntityGraph(attributePaths = "provider")`. No `ServiceOfferingService`, a classe toda é `@Transactional` e a busca é `@Transactional(readOnly = true)`; ela devolve `repository.findAll(spec, pageable).map(this::toResponseDTO)`, ou seja, o DTO é montado dentro da transação.

O `toResponseDTO` lê `service.getProvider().getFullName()`. Com `open-in-view=false`, isso só funciona dentro do método transacional, e só não gera N+1 graças ao `@EntityGraph`. O SQL sai pelo logger `org.hibernate.SQL`, e não por `show-sql`, para cada linha levar o id da requisição.

Erros e teste

O `GlobalExceptionHandler` estende `ResponseEntityExceptionHandler` e tem um `@ExceptionHandler` por exceção de domínio; os `type` ficam num lugar só, a classe `ProblemTypes`, como `URI.create("urn:servicehub:problem:not-found")`. O último handler, para `Exception`, registra o erro no log e responde 500 com o `detail` "Erro interno. Informe o X-Request-Id ao suporte." A decisão está na ADR-0004.

O teste de fatia web `ServiceOfferingControllerTest` é anotado com `@WebMvcTest(ServiceOfferingController.class)` e `@AutoConfigureMockMvc(addFilters = false)` (os filtros de segurança só chegam na Semana 08) e declara `@MockitoBean ServiceOfferingService service`. O caso `deveRetornar400QuandoPrecoZero` faz `POST /api/v1/services` com preço 0 e espera `status().isBadRequest()` e `jsonPath("$.errors[0].field").value("price")`. Outro caso envia a categoria `INVALIDA` e confere que o `detail` lista as categorias aceitas.

14. Construindo com IA

↑ conteúdo

- **Especificar:** entregue o trecho exato de `servicehub_schema.sql` e o contrato (rota, status, formato da página).
- **Gerar:** plano primeiro (arquivos, migração, colunas e tipos); depois o código, migração antes de Java.
- **Revisar:** leia o diff da migração antes do Java; confira coluna por coluna, `jakarta` e `EnumType.STRING`.
- **Provar:** a aplicação sobe com `validate` no Postgres do Compose e uma chamada real devolve a página do Mapa.

Prompt fraco e prompt bom

Fraco

```
cria a entidade de serviços com JPA e o CRUD
```

A IA escolhe colunas, tipos e versões. Costuma vir `ddl-auto=update` e a entidade na resposta.

Bom

Diz qual migração criar e a partir de que arquivo, proíbe editar as existentes, pede plano e termina com um aceite verificável. Está completo logo abaixo.

O prompt desta semana

Ponto de partida: a `etapa-1` (com o `InfoController`) mais o primeiro passo da aula: starters de persistência no `pom.xml`, Postgres no Compose, `V1__Create_users_table.sql` e a entidade `User`.

```
Contexto: API ServiceHub, Java 21, Spring Boot 4.1, pacote base
com.servicehub; starters webmvc, data-jpa, flyway (+ flyway-database-
postgresql), validation e actuator; PostgreSQL 16 no Compose em
localhost:5433. Já existem V1__Create_users_table.sql e a entidade User.
Schema final: docs/alunos/servicehub_schema.sql (anexo a tabela services).

Objetivo: tabela services e listagem pública paginada GET /api/v1/services.

Entregáveis:
1. db/migration/V2__Create_services_table.sql igual à tabela services
   do schema (tipos, NOT NULL, DEFAULT, CHECKs, FK RESTRICT, índices).
2. Enum ServiceCategory com os 21 valores do CHECK, mesma grafia.
3. Entidade ServiceOffering em com.servicehub.model.
4. ServiceOfferingRepository com @EntityGraph no prestador (sem N+1).
5. Record ServiceOfferingResponse: id, title, description, price,
   category, active, providerId, providerName, createdAt, updatedAt.
6. Service @Transactional(readOnly = true) que monta o record dentro
```

```
   da transação; controller com @PageableDefault(size = 20).

Restrições:
- NÃO edite nem renomeie a V1. Só crie a V2. SQL do PostgreSQL.
- ddl-auto continua validate (nunca update); open-in-view=false.
- jakarta.persistence, nunca javax. @Enumerated(EnumType.STRING).
- price é BigDecimal; @ManyToOne(fetch = LAZY) para o prestador.
- O controller NUNCA devolve a entidade; só o record.
- Paginação: max-page-size=100 e serialization-mode=via-dto nas
  propriedades. NÃO use @EnableSpringDataWebSupport.
- Sem endpoints de escrita neste passo.

Antes do código, mostre um plano: arquivos, o SQL da V2 e o
mapeamento coluna a coluna (coluna, tipo SQL, atributo, tipo Java).
Espere minha aprovação.

Critério de aceite: a aplicação sobe sem erro de validação do
Hibernate, o log do Flyway mostra a versão 2 aplicada e
GET /api/v1/services responde 200 com página vazia:
{"content":[],"page":{"size":20,"number":0,
 "totalElements":0,"totalPages":0}}
```

O que a IA costuma errar nesta etapa

- Sugerir `ddl-auto=update` "para facilitar", ou editar a V1 em vez de criar uma versão nova.
- Código de Boot antigo (`javax.persistence`, `@MockBean`, `flyway-core` sem starter) ou `@EnableSpringDataWebSupport`, que desliga o teto de 100.
- Nome de coluna divergente da migração, às vezes com um espaço invisível (`" provider_id"`), `EnumType.ORDINAL` ou `double` no preço.
- Controller devolvendo a entidade, sem paginação, ou DTO montado fora da transação.

Checklist de revisão

☐ A V1 está intacta no `git diff`? Só a V2 é nova?

☐ Cada coluna da V2 bate com o schema dos alunos e com um atributo da entidade?

☐ O controller devolve o record, e não a entidade?

☐ As três propriedades de paginação e o `open-in-view=false` estão no `application.properties`?

Prova

```
docker compose up -d postgres
mvnw.cmd spring-boot:run
curl -i http://localhost:8080/api/v1/services
curl -i "http://localhost:8080/api/v1/services?size=500"
```

Esperado: 200 com a página vazia do aceite e `"size":100` na segunda chamada; no `psql`, `flyway_schema_history` lista as duas migrações.

15. Glossário

↑ conteúdo

Impedância objeto-relacional	Diferença entre o modelo de objetos e o modelo relacional, que exige tradução.
ORM	Ferramenta que mapeia classes para tabelas e gera o SQL de leitura e gravação.
JPA	Jakarta Persistence: especificação de anotações e interfaces de persistência em Java.
Contexto de persistência	Conjunto de entidades gerenciadas numa transação, com uma instância por id.
Dirty checking	Comparação automática que transforma mudanças em entidades gerenciadas em `UPDATE`.
LAZY / EAGER	Estratégias de carga de uma associação: sob demanda ou sempre junto.
N+1	Uma consulta para a lista e mais uma para cada item, ao acessar associações LAZY.
Transação	Grupo de operações tratado como unidade, com as garantias ACID.
Migração	Script versionado que leva o schema de uma versão à seguinte.
Checksum	Número calculado do conteúdo de um arquivo; muda se o arquivo muda.
Repository	Interface que se comporta como coleção de objetos do domínio.
DTO	Objeto só com dados, que atravessa a fronteira da API no lugar da entidade.
Mass assignment	Ataque em que o cliente envia campos que não deveria poder definir.
ProblemDetail	Formato padronizado de erro HTTP da RFC 9457 (`application/problem+json`).

16. Exercícios

↑ conteúdo

1. Explique a diferença entre JPA e Hibernate. Por que o código das entidades importa `jakarta.persistence` e não `org.hibernate`?
2. Num método `@Transactional`, o código busca um serviço com `findById`, muda o preço e termina sem chamar `save()`. O preço é gravado? Explique usando os termos "managed" e "dirty checking". E se o mesmo objeto fosse alterado depois do fim da transação?
3. Uma listagem de 50 ordens mostra o nome do cliente de cada uma, e o log mostra 51 `SELECT`s. Dê o nome do problema, a causa e duas formas de corrigi-lo.
4. Um colega editou a `V2__Create_services_table.sql`, já aplicada, para acrescentar a coluna `updated_at`. O que acontece na próxima subida e qual é a correção certa?
5. Compare `ddl-auto=update` e `ddl-auto=validate` num projeto com Flyway. Por que o ServiceHub usa `validate`?

6. O enum `Situacao` tem `DISPONIVEL, EMPRESTADO` e é mapeado com `EnumType.ORDINAL`. Alguém insere `RESERVADO` entre os dois. O que acontece com as linhas já gravadas?
7. Analise o código: `@GetMapping("/{id}") public User buscar(@PathVariable Long id) { return repo.findById(id).orElseThrow(() -> new RuntimeException("não achei")); }`. Aponte três problemas em relação ao contrato do ServiceHub e reescreva em linhas gerais.
8. Por que, no ServiceHub, as exceções de negócio estendem `RuntimeException`? Relacione com o comportamento padrão de rollback do `@Transactional`.
9. Monte o corpo ProblemDetail que a API deve devolver para `POST /api/v1/services` com `"price": 0`. Indique status, `type` e o conteúdo de `errors`.
10. Um cliente pede `GET /api/v1/services?size=500`. O que a API responde e por que esse teto é uma regra de segurança? Que anotação, se adicionada, faria o teto sumir?
11. Escreva um prompt, seguindo o formato da seção 14, para pedir à IA a migração e a entidade da tabela `reviews`. Inclua pelo menos três restrições e um critério de aceite verificável.

17. Para estudar

↑ conteúdo

Leituras disponíveis na Minha Biblioteca, pelo acesso institucional:

- RODRIGUES, T. N. et al. *Integração de aplicações*. Grupo A, 2020. Capítulo "Servidores de aplicação JPA".
- MACHADO, R. P.; FRANCO, M. H. I.; BERTAGNOLLI, S. C. *Desenvolvimento de software III: programação de sistemas web orientada a objetos em Java*. Bookman (Tekne), 2016. Capítulo 4.
- FOWLER, M. *Padrões de arquitetura de aplicações corporativas*. Bookman, 2006. Capítulos 3 (mapeamento para bancos de dados relacionais) e 10 (padrões de fonte de dados: Active Record e Data Mapper).
- SCHILDT, H. *Java para iniciantes*. 6. ed. Bookman. Capítulo 9 (exceções).
- MARTIN, R. C. *Código limpo*. Alta Books, 2009. Capítulo 7 (tratamento de erros).

Referências abertas:

- IETF. RFC 9457: *Problem Details for HTTP APIs*. 2023. Leia até a seção 3 (os membros do objeto).
- Documentação oficial do Spring Data JPA: `spring.io/projects/spring-data-jpa`.
- Documentação oficial do Flyway: `flywaydb.org`.
- Documentação oficial do Spring Boot, capítulos de acesso a dados e Actuator: `docs.spring.io/spring-boot`.